

Sistemas Operacionais e Computação em Nuvem

Entrega 2

Aether AI

Bruno Da Silva Ribeiro 24025958

Kauan Rocha Dias 24026492

Gabriel Henrique Coelho Marussi 24026609

Arthur Rodrigues Ferreira 24026567

1. Preparando o ambiente

Para iniciar o ambiente de desenvolvimento e execução dos scripts, foram instaladas as seguintes bibliotecas Python essenciais:

```
Pichau@DESKTOP-M2QG9MP MINGW64 ~/Desktop/monitoramento-ia  
o $ pip install pandas matplotlib seaborn scikit-learn psutil numpy streamlit
```

Essa etapa garante que todas as dependências necessárias para o monitoramento, análise de dados e visualização estejam disponíveis no sistema

2. Script de Monitoramento (monitor.py)

O script monitor.py é responsável por coletar dados de uso da CPU, memória e disco do sistema em intervalos regulares e armazená-los em um arquivo CSV. Isso permite o acompanhamento do desempenho do sistema ao longo do tempo.

```

monitor.py > ...
1  import psutil
2  import pandas as pd
3  import time
4  from datetime import datetime
5  import os
6
7  arquivo = 'dados_monitoramento.csv'
8
9  while True:
10
11     dados = {
12         'timestamp': [datetime.now()],
13         'cpu': [psutil.cpu_percent()],
14         'memoria': [psutil.virtual_memory().percent],
15         'disco': [psutil.disk_usage('/').percent]
16     }
17
18     df_novo = pd.DataFrame(dados)
19
20     if os.path.exists(arquivo):
21         df_antigo = pd.read_csv(arquivo)
22         df_final = pd.concat([df_antigo, df_novo], ignore_index=True)
23     else:
24         df_final = df_novo
25
26     df_final.to_csv(arquivo, index=False)
27
28     print(df_novo)
29
30     time.sleep(5)

```

O script monitor.py utiliza as bibliotecas psutil para coletar métricas de CPU, memória e disco do sistema. Em um loop contínuo, ele registra o timestamp e os percentuais de uso desses recursos, armazenando-os em um DataFrame do pandas. Os dados são periodicamente salvos em um arquivo CSV (dados_monitoramento.csv), com a funcionalidade de adicionar novas entradas a um arquivo existente ou criar um novo se necessário. A cada 5 segundos, o script exibe os dados mais recentes no console e pausa, garantindo um monitoramento regular do desempenho do sistema

Exemplo de monitoramento com o PC ocioso:

	timestamp	cpu	memoria	disco
0	2026-05-09 04:40:16.504516	15.4	59.7	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:40:21.514443	6.1	59.7	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:40:26.531150	6.0	59.7	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:40:31.544826	4.0	59.6	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:40:36.556509	7.5	59.6	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:40:41.575068	7.5	59.7	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:40:46.587003	4.7	59.6	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:40:51.601009	6.8	59.6	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:40:56.617202	6.5	59.6	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:41:01.636831	3.9	59.6	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:41:06.656388	6.9	59.6	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:41:11.669607	8.3	59.7	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:41:16.682674	5.9	59.7	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:41:21.695082	6.2	59.7	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:41:26.711341	6.1	59.7	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:41:31.722022	8.8	59.7	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:41:36.734796	7.1	59.6	95.3
	timestamp	cpu	memoria	disco
0	2026-05-09 04:41:41.747611	11.4	59.5	95.3

3. Teste de Estresse

Para simular cenários de alta demanda e analisar o comportamento do sistema sob carga elevada, foi desenvolvido um script de teste de estresse utilizando Python.

```

stress.py > ...
1  import multiprocessing
2
3  def estressar_cpu():
4      while True:
5          x = 0
6          for i in range(100000000):
7              x += i * i
8
9  if __name__ == "__main__":
10
11      processos = []
12
13      for _ in range(multiprocessing.cpu_count()):
14
15          p = multiprocessing.Process(target=estressar_cpu)
16
17          p.start()
18
19          processos.append(p)
20
21      for p in processos:
22          p.join()

```

O arquivo stress.py foi implementado utilizando a biblioteca multiprocessing, permitindo a criação de múltiplos processos paralelos para utilização intensiva da CPU. A função estressar_cpu() executa cálculos matemáticos repetitivos em um loop infinito, aumentando significativamente o uso do processador. O número de processos criados é baseado na quantidade de núcleos disponíveis no computador, permitindo que o teste utilize múltiplos núcleos simultaneamente. Durante a execução do teste de estresse, o sistema de monitoramento permaneceu ativo coletando informações de CPU, memória e disco em tempo real. Isso possibilitou observar claramente as mudanças de comportamento do sistema durante períodos de alta utilização. O encerramento do teste foi realizado manualmente utilizando CTRL + C no terminal, interrompendo os processos de estresse após a coleta dos dados necessários.

```

Pichau@DESKTOP-M2QG9MP MINGW64 ~/Desktop/monitoramento-ia
$ python stress.py

```

Exemplo de monitoramento com o PC em estresse:

```
0 2026-05-09 04:52:44.318172 100.0 62.7 95.3
timestamp cpu memoria disco
0 2026-05-09 04:52:49.586407 100.0 62.8 95.3
timestamp cpu memoria disco
0 2026-05-09 04:52:54.617604 100.0 62.8 95.3
timestamp cpu memoria disco
0 2026-05-09 04:52:59.649022 100.0 62.6 95.3
timestamp cpu memoria disco
0 2026-05-09 04:53:04.689236 100.0 62.6 95.3
timestamp cpu memoria disco
0 2026-05-09 04:53:09.721548 100.0 62.5 95.3
timestamp cpu memoria disco
0 2026-05-09 04:53:15.776105 100.0 62.5 95.3
timestamp cpu memoria disco
0 2026-05-09 04:53:20.818731 100.0 62.5 95.3
timestamp cpu memoria disco
0 2026-05-09 04:53:25.868452 100.0 62.5 95.3
timestamp cpu memoria disco
0 2026-05-09 04:53:31.133778 100.0 62.5 95.3
timestamp cpu memoria disco
0 2026-05-09 04:53:36.494424 100.0 62.5 95.3
timestamp cpu memoria disco
0 2026-05-09 04:53:41.825146 100.0 62.5 95.3

```

4. Análise de Dados com Inteligência Artificial (ia.py)

Para a análise dos dados coletados e a identificação de padrões e anomalias, foi desenvolvido o script ia.py, que implementa técnicas de Regressão Linear, K-Means e Isolation Forest.

```
ia.py > -
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 from sklearn.linear_model import LinearRegression
6 from sklearn.cluster import KMeans
7 from sklearn.ensemble import IsolationForest
8
9 # Ler dados
10 df = pd.read_csv('dados_monitorados.csv')
11
12 # -----
13 # REGRESSÃO LINEAR
14 # -----
15
16 df['tempo'] = np.arange(len(df))
17
18 X = df[['tempo']]
19 y = df['cpu']
20
21 modelo = LinearRegression()
22 modelo.fit(X, y)
23
24 previsao = modelo.predict([[len(df)+10]])
25
26 print('Previsão futura CPU:')
27 print(previsao)
28
29 # Gráfico regressão
30
31 plt.figure(figsize=(10,5))
32
33 plt.plot(df['tempo'], df['cpu'], label='CPU Real')
34
35 plt.plot(
36     df['tempo'],
37     modelo.predict(X),
38     label='Regressão Linear'
39 )
40
41 plt.legend()
42
43 plt.title('Regressão Linear CPU')
44
```

```

# -----
# K-MEANS
# -----

X_cluster = df[['cpu', 'memoria']]

kmeans = KMeans(n_clusters=3)

kmeans.fit(X_cluster)

df['cluster'] = kmeans.labels_

plt.figure(figsize=(8,6))

plt.scatter(
    df['cpu'],
    df['memoria'],
    c=df['cluster']
)

plt.xlabel('CPU')
plt.ylabel('Memória')

plt.title('Clusters K-Means')

plt.savefig('grafico_kmeans.png')

# -----
# ISOLATION FOREST
# -----

iso = IsolationForest(contamination=0.05)

df['anomalia'] = iso.fit_predict(X_cluster)

plt.figure(figsize=(10,5))

plt.plot(df['cpu'], label='CPU')

anomalias = df[df['anomalia'] == -1]

plt.scatter(
    anomalias.index,
    anomalias['cpu']
)

```

```

plt.legend()

plt.title('Detecção de Anomalias')

plt.savefig('grafico_anomalias.png')

print('Anomalias detectadas:')
print(anomalias)

```

```

Pichau@DESKTOP-P2Q69MP MINGW64 ~/Desktop/monitoramento-ia
$ python ia.py
C:\Users\Pichau\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature
es
  warnings.warn(
Previsão futura CPU:
[76.10141195]
Anomalias detectadas:
  timestamp      cpu  memoria  disco  tempo  cluster  anomalia
4  2026-05-09 04:38:46.000167  34.5    61.6   95.3      4         2        -1
14 2026-05-09 04:39:38.505204  100.0   61.9   95.3     14         1        -1
44 2026-05-09 04:42:31.927119   46.0    60.6   95.3     44         2        -1

```

O script ia.py processa os dados de monitoramento (dados_monitoramento.csv) aplicando três técnicas de Machine Learning para análise de desempenho do sistema:

Regressão Linear: Utilizada para prever tendências de uso da CPU, gerando um gráfico comparativo entre o uso real e a previsão.

K-Means: Agrupa os estados de CPU e memória em clusters, identificando padrões de operação como ocioso, normal ou sobrecarga, visualizados em um gráfico de dispersão.

Isolation Forest: Detecta anomalias no uso da CPU, destacando eventos inesperados em um gráfico e listando seus detalhes.

Cada análise resulta na geração de gráficos específicos para visualização dos resultados.

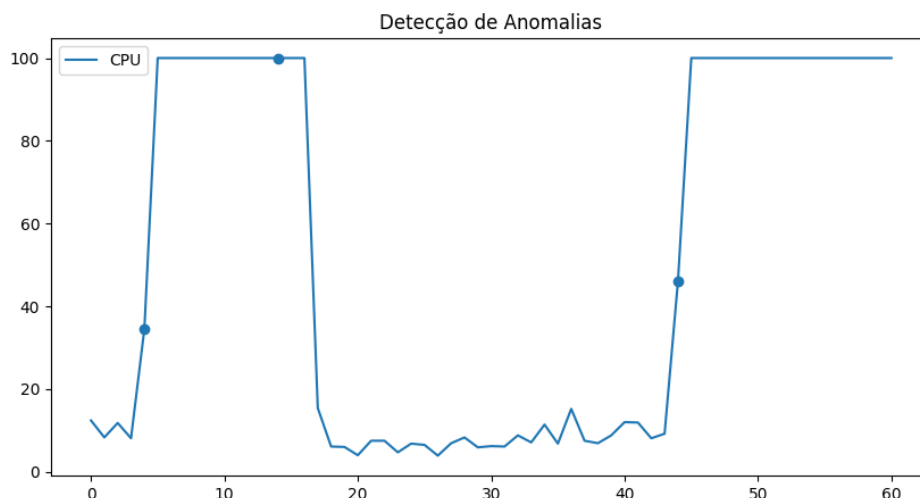
5. Aplicação da Inteligência Artificial

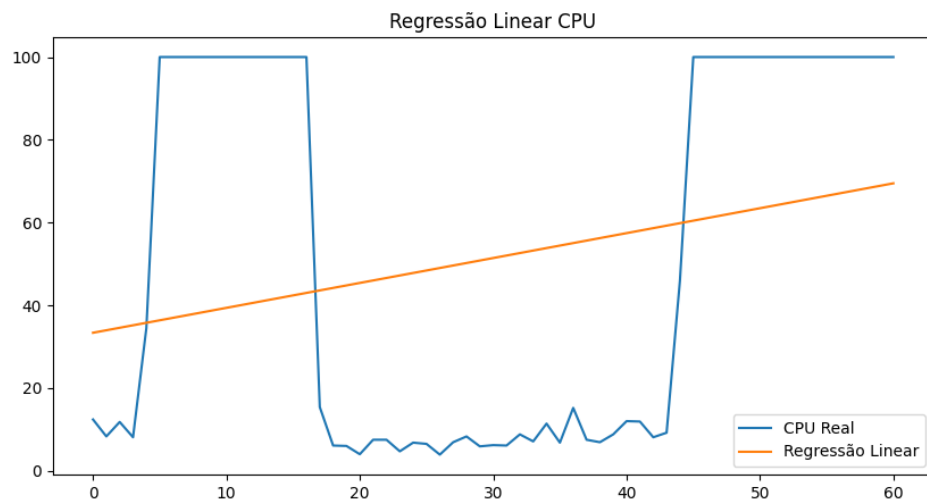
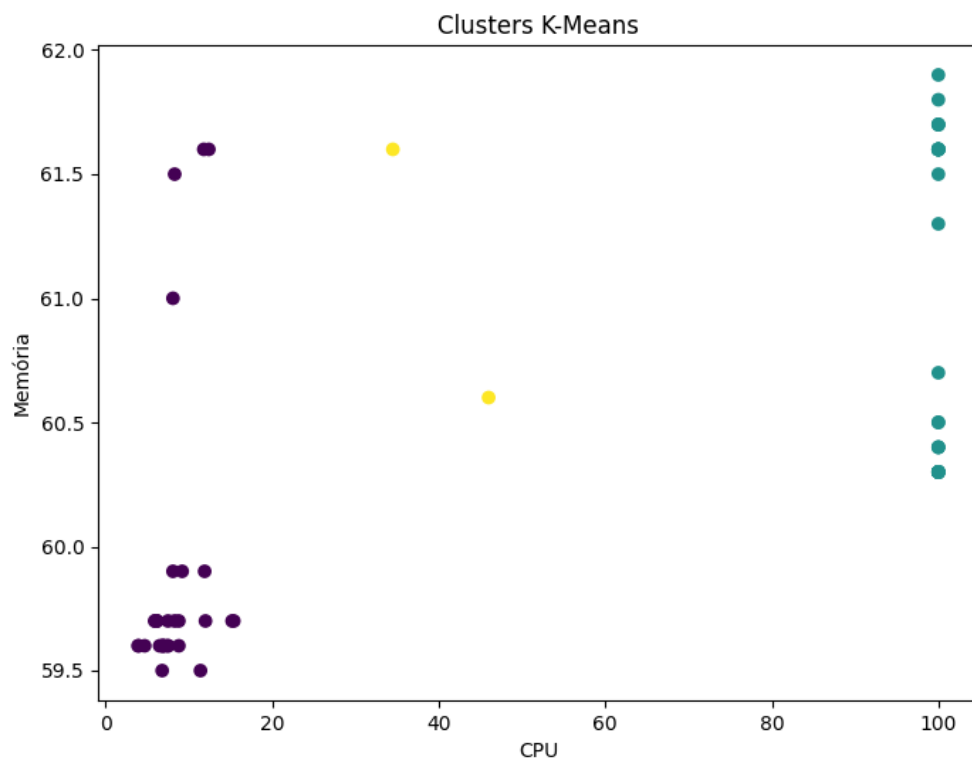
A aplicação de Inteligência Artificial neste projeto demonstra como algoritmos de Machine Learning podem transformar dados brutos de telemetria em insights acionáveis para o gerenciamento de infraestrutura. O objetivo principal foi ilustrar a capacidade desses modelos de processar informações do sistema operacional em tempo real, auxiliando em decisões arquitetônicas e operacionais. A estratégia foi dividida em três pilares fundamentais:

Análise de Tendência (Supervisionada - Regressão Linear): Utilizada para prever o comportamento futuro do consumo de recursos (CPU). Em cenários reais, isso é crucial para o Auto Scaling preditivo, permitindo que o sistema provisione recursos proativamente antes que limites críticos sejam atingidos.

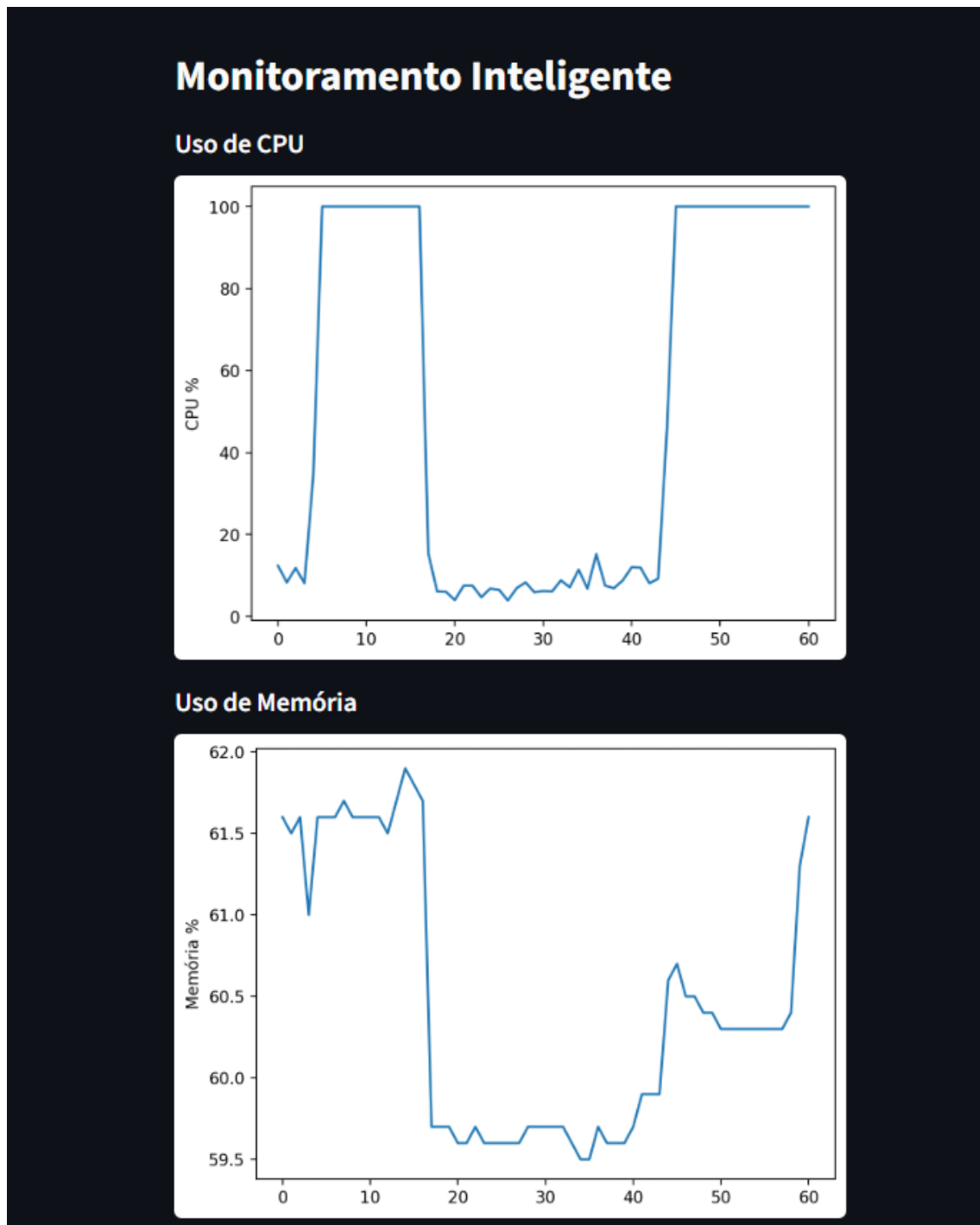
Identificação de Perfis (Não-Supervisionada - K-Means): Aplicada para classificar dinamicamente a "saúde" ou o estado do servidor, agrupando padrões de uso de CPU e memória. Isso ajuda a identificar perfis de carga de trabalho e otimizar a alocação de instâncias na nuvem.

Segurança e Resiliência (Não-Supervisionada - Isolation Forest): Empregada para detectar comportamentos anômalos que desviam do padrão normal de operação. Picos repentinos ou quedas abruptas podem indicar falhas, ataques ou gargalos, permitindo alertas imediatos para as equipes de operação.





Um pequeno Dashboard mostrando o uso da CPU:



6. Análise e Interpretação dos Resultados

Os resultados obtidos através da execução dos scripts e da geração dos gráficos confirmam a eficácia das abordagens escolhidas.

Regressão Linear: O modelo foi capaz de traçar uma linha de tendência ascendente durante o período de teste de estresse, indicando claramente o aumento contínuo da carga no servidor.

K-Means: O gráfico de dispersão gerado pelo K-Means revelou a formação de clusters distintos. É possível observar agrupamentos que representam o estado ocioso (baixo uso de CPU e memória) e o estado de estresse (alto uso de CPU), validando a capacidade do algoritmo de classificar os estados do sistema.

Isolation Forest: O gráfico de detecção de anomalias destacou com sucesso os pontos onde o uso da CPU sofreu variações abruptas, especialmente no início e no fim do teste de estresse. A saída no console confirmou a identificação desses eventos anômalos, registrando os timestamps exatos em que ocorreram.

7. Conclusão

Além disso, o projeto demonstrou como técnicas de Inteligência Artificial podem auxiliar na automação da análise de desempenho computacional, permitindo identificar padrões e detectar anomalias de forma eficiente. Os resultados obtidos validam a utilização de Machine Learning como ferramenta de apoio ao monitoramento moderno de sistemas.